

IntraComms

Messaging & Security Protocol Architecture

Project:	LAN Offline Messaging & File Sharing System
Author:	Ibrahim Habib - Lincoln University College
Stack:	Flask / Socket.IO / Web Crypto API / AES-256-GCM
Version:	2025 - Final Year Project

CONFIDENTIAL - Internal use only

1. System Overview

IntraComms is a LAN-only, offline-first group/direct messaging system. It has no cloud dependency - all traffic stays inside your local network. The server is a single Python process (server/server.py) built on Flask + Flask-SocketIO + eventlet. The client is a browser page with raw WebSocket handling and the Web Crypto API.

Architecture Diagram



Key components:

server/server.py	All routes, WebSocket handlers, E2EE key endpoints
server/models/	SQLAlchemy ORM (User, Message, FileRecord)
static/js/chat.js	EIO4 client, E2EE module, file upload
static/css/app.css	Single stylesheet - all tokens in :root
uploads/	Processed media files (unencrypted at rest)
intracomms.db	SQLite database

Technology Stack

Layer	Technology
HTTP server	Flask 3.x + Werkzeug
WebSocket	Flask-SocketIO + eventlet (EIO4 transport)
Database	SQLite via SQLAlchemy ORM
Auth	Flask-Login + Werkzeug PBKDF2-SHA-256

TLS

Self-signed cert (cryptography lib or openssl CLI)

E2EE (client)

Web Crypto API - ECDH P-256 + AES-256-GCM

Media processing

Pillow (images) + FFmpeg (audio/video)

Scheduling

APScheduler (stale upload cleanup)

2. Transport Security (TLS / HTTPS)

Certificate Generation

At first startup, `_ensure_ssl_cert()` runs two fallback methods to produce `cert.pem` and `key.pem` in the project root. The cert includes a Subject Alternative Name (SAN) with `localhost`, `127.0.0.1`, and the machine's LAN IP.

Method	Library	Key Type	Validity
1 (primary)	cryptography Python package	RSA-2048	10 years
2 (fallback)	openssl CLI (MSYS2/Git Bash)	RSA-2048	10 years

Why TLS Is Required

`window.crypto.subtle` (the Web Crypto API) is only available in a secure context - HTTPS or localhost. Any LAN client on `http://192.168.x.x:5000` has `crypto.subtle = undefined`, which completely disables E2EE. HTTPS is not optional for multi-device use.

localStorage Origin Isolation - Critical

Chrome isolates `localStorage` by origin. `http://host:5000` and `https://host:5000` are completely separate namespaces. Switching protocols invalidates all stored private keys. If you change HTTP to HTTPS, delete `intracomms.db` and let all users re-register fresh keypairs.

3. Authentication & Session Management

Mechanism	Detail
Password hashing	werkzeug PBKDF2-SHA-256 (<code>generate_password_hash</code>)
Session cookie	Flask-Login signed cookie (<code>SECRET_KEY</code> from env)
WebSocket auth	<code>current_user.is_authenticated</code> checked on every connect event
First user	Automatically promoted to admin role
Admin CLI	<code>flask promote-admin <username></code>
Account control	Admin can activate / deactivate / change role

All routes that serve data are decorated with `@login_required`. The WebSocket `handle_connect` handler rejects unauthenticated connections by returning `False`.

4. Real-Time Transport (WebSocket / EIO4)

The client implements the Engine.IO v4 protocol manually in chat.js. There is no socket.io.js library bundled - the framing is done in raw JS.

Handshake Sequence

```
Client          Server
|-- WSS connect ----->|
|<-- '0' (EIO open) ---| Server sends EIO open packet
|-- '40' ----->| Client sends Socket.IO connect
|<-- '40' -----| Server confirms Socket.IO connected
                  <- flushQueue() runs here (queued msgs sent)
```

Keep-Alive (Ping / Pong)

```
Server --- '2' (ping) ---> Client
Client <-- '3' (pong) --- Server
```

Message Framing

All application messages use prefix "42" (Socket.IO event packet):

```
42["message", {
  "type": "send_message",
  "payload": { "receiver_id": "5", "content": "<ciphertext>", ... },
  "requestId": "abc123"
}]
```

Room Strategy (Server-Side)

Room	Members	Used For
------	---------	----------

user_{id}

Each user on connect

Direct messages, presence events

broadcast

All users on connect

Broadcast messages, presence broadcast

Reconnect Strategy

Exponential backoff: starts at 1 s, doubles on each failure, caps at 30 s. Messages sent while disconnected are buffered in `messageQueue[]` and flushed automatically after reconnect.

5. End-to-End Encryption (E2EE) Protocol

Algorithm Suite

Component	Algorithm	Notes
Identity keys	ECDH P-256	Persistent per-device, stored as JWK in localStorage
Key agreement	ECIES (ephemeral DH)	New ephemeral keypair per message, discarded after 128-bit random IV; provides confidentiality +
Symmetric cipher	AES-256-GCM	One-step HKDF-like derivation
Key derivation	SubtleCrypto deriveKey(ECDH)	Browser CSPRNG; Math.random never used
PRNG	crypto.getRandomValues	

Key Lifecycle

```

FIRST VISIT (new device or cleared localStorage)
  crypto.subtle.generateKey(ECDH P-256)
    |- privateKey --> persisted as JWK in localStorage['intracomms_priv_jwk_v1']
    \- publicKey --> base64-SPKI --> POST /api/me/pubkey --> User.public_key in DB

SUBSEQUENT VISITS (same device, same origin)
  load JWK from localStorage
    |- importKey --> _privateKey (non-extractable CryptoKey in memory)
    \- reconstruct publicKey --> re-POST /api/me/pubkey (idempotent on server)

```

Encryption Flow (Sender --> Recipient)

1. Fetch recipient public key (base64-SPKI) from userPubkeys[] cache
If not cached --> GET /api/users/<id>/pubkey
2. Generate ephemeral ECDH P-256 keypair -- discarded after this call
3. DH(ephemeral_private, recipient_public) --> shared_secret_1

deriveKey(AES-256-GCM 256-bit) from shared_secret_1

```
iv_1 = crypto.getRandomValues(12 bytes)
```

```
ciphertext_for_recipient = AES-GCM-Encrypt(key_1, iv_1, plaintext)
```


4. ALSO encrypt for own public key (so sender can read own messages):

Generate second ephemeral keypair

$\text{DH}(\text{ephemeral_private_2}, \text{sender_public}) \rightarrow \text{shared_secret_2}$

```
sender_copy = AES-GCM-Encrypt(key_2, iv_2, plaintext)
```


5. Send via WebSocket:

```
{ content: ciphertext_for_recipient,  ephemeral_pub: ep1,  iv: iv1,
```

sender_copy: ciphertext_for_sender, sender_ephemeral_pub: ep2,


```
sender_iv: iv2, is_e2ee: true }
```

Server Storage (Zero Knowledge)

The server stores ciphertext as-is and never sees plaintext for E2EE messages.

DB Column	Contents
content	Recipient's ciphertext (base64)
ephemeral_pub	Ephemeral public key for recipient key derivation
iv	AES-GCM nonce for recipient (base64)
sender_copy	Sender's own ciphertext (base64)
sender_ephemeral_pub	Ephemeral public key for sender key derivation
sender_iv	AES-GCM nonce for sender (base64)
is_e2ee	Boolean flag - controls client decrypt path

Decryption Flow (Receiver's Browser)

1. Receive new_message envelope via WebSocket
2. Detect is_e2ee=true and message_type='direct'
3. Select correct ciphertext pair:
Sender reads: sender_copy + sender_ephemeral_pub + sender_iv
Recipient reads: content + ephemeral_pub + iv
4. importKey(ephemeral_pub, SPKI) --> CryptoKey
5. deriveKey(ECDH, ephemeral_pub, _privateKey) --> AES-256-GCM key
6. AES-GCM decrypt(iv, ciphertext) --> plaintext
7. Display in message bubble

AES-GCM authentication: any tamper causes decrypt() to throw,

producing 'Unable to decrypt message' in the UI.

Historical Messages (Page Load)

On dashboard load the server renders ciphertext into data-ct, data-ep, data-iv HTML attributes. After E2EE.init() resolves, decryptHistorical() walks every .e2ee-pending element and decrypts in-place.

What Is NOT Encrypted

- Broadcast messages - always plaintext (public by nature; per-recipient E2EE would require a ratchet protocol).
- File uploads - FileRecord.ephemeral_pub stores 'plain-text-prototype'; files are unencrypted on disk.
- Metadata - sender ID, receiver ID, timestamps, filenames always stored in plaintext.

6. Complete Message Flow

Primary Path (WebSocket + E2EE)

```
[User presses Enter]
|
|- receiverId == 'broadcast'?
|   \- sendWs() --> plaintext over WebSocket
|
\ - DM?
    |- pubkey in cache? YES --> _sendEncrypted()
    \ - NO --> GET /api/users/<id>/pubkey --> cache --> _sendEncrypted()
        |
        v
        E2EE.encryptFor(recipientPub) + E2EE.encryptFor(myPub)
        |
        v
        socket.send('42[message, {type:send_message, payload:{...}}]')
        |
        [SERVER] handle_socket_message()
        |- rate limit check (30/min per user)
        |- build_message() -- validate receiver
        |- db.session.add(message) + commit
        \ - emit_saved_message()
            |
            |- emit --> room 'user_{sender_id}'
            \ - emit --> room 'user_{receiver_id}'
                |
                dispatch() --> appendMessage()
                |
                E2EE.decrypt() --> display
```

HTTP Fallback Path

If WebSocket is unavailable, the form submits as POST /messages. The server saves the message and redirects to the dashboard. This path always sends plaintext - no E2EE.

7. File Transfer

Standard Files (Images, Audio, Documents)

```
POST /upload (multipart FormData)
|- extension whitelist check
|- size limit check (max 50 MB)
|- image/ --> Pillow: resize, strip EXIF, convert to WebP + thumbnail
|- audio/ --> FFmpeg: re-encode to Opus OGG
\ - other --> save as-is
FileRecord + Message saved --> emit_saved_message() --> both parties notified
```

Video (Chunked Upload)

Large videos are split into chunks (size determined by server-reported video_chunk_size based on connection tier) and sent sequentially to POST /upload/video/chunk. When all chunks arrive, a background thread:

- Assembles chunks into a single raw file
- Extracts a poster frame with ffmpeg for the thumbnail
- Transcodes to H.264/AAC MP4 with ffmpeg
- Updates FileRecord.status from 'processing' to 'ready'
- Emits media_ready event to notify both parties instantly

Adaptive Quality (Connection Tiers)

On every WebSocket connect the client measures RTT with GET /ping and reports a tier via the client_tier socket event.

Tier	RTT	Downlink	Photo max	Video CRF	Scale
wired	< 5 ms	> 50 Mbps	2560 px	23	1080p
wifi_good	< 20 ms	> 20 Mbps	1920 px	25	720p
wifi_weak	< 60 ms	> 5 Mbps	1280 px	28	480p
slow	>= 60 ms	<= 5 Mbps	800 px	30	360p

The server uses the worst tier across all active sessions of a user (multi-tab safe via `client_tiers` dict keyed by socket session ID).

File Access Control

Check	Rule
Authentication	@login_required on all /uploads/* routes
Ownership	uploader_id == current_user.id
Direct share	receiver_id == current_user.id
Broadcast files	visibility == 'broadcast'
Deleted files	404 returned regardless of ownership
Processing	202 returned with retry_after:5 while transcoding

8. Presence System

Server-Side Data Structures

```
_sid_to_uid: dict # socket session ID --> user ID
_uid_conns: dict # user ID --> active connection count (multi-tab safe)
```

Presence Events

Event	Direction	Trigger	Payload
online_users	Server -> connecting	On WebSocket connect	{user_ids: [1, 3, 7]} - full snapshot
user_online	Server -> broadcast	First connection for a	{user_id: 5}
user_offline	Server -> broadcast	Last tab closed	{user_id: 5}
user_joined	Server -> broadcast	New user registers	{user_id, username, email, color_index,

Multi-tab safety: `_uid_conns[uid]` is a reference count. `user_offline` is only emitted when the count drops to zero (all tabs closed). `user_online` is only emitted when count rises from zero to one (first tab opened).

9. Rate Limiting

Server-side only, stored in the `_rate_limit` in-memory dict. Limit: 30 messages per 60-second window per user. Exceeded requests receive a `RATE_LIMITED` error event over the socket. Stale past-minute windows are pruned on each check to prevent memory growth.

```
_rate_limit: dict # (user_id, minute_window) --> count

window = int(time.time() // 60) # current 1-minute bucket
key = (user_id, window)
if _rate_limit.get(key, 0) >= 30: reject with RATE_LIMITED
```

10. Authorization Model

Action	Requirement
View any page	@login_required - redirects to login if not authenticated
Send a direct message	Authenticated + receiver must be active + not self
Send broadcast	Authenticated - any active user
Delete a message	Own messages only (msg.sender_id == current_user.id)
Download a file	Uploader OR receiver OR broadcast visibility
Admin dashboard	current_user.is_admin() (role == 'admin')
Deactivate/activate user	Admin only - cannot deactivate own account
Change user role	Admin only - cannot self-demote

11. Database Schema

users

Column	Type	Notes
id	INTEGER PK	Auto-increment
username	VARCHAR(80)	Unique
email	VARCHAR(120)	Unique, lowercase
password_hash	VARCHAR(256)	PBKDF2-SHA-256
role	VARCHAR(20)	'admin' or 'user'
is_active	BOOLEAN	Default True; admin-controlled
public_key	TEXT	base64-SPKI ECDH P-256 public key
created_at	DATETIME	UTC timestamp

messages

Column	Type	Notes
id	INTEGER PK	
sender_id	INTEGER FK	-> users.id
receiver_id	INTEGER FK	-> users.id; NULL for broadcast
content	TEXT	Ciphertext (E2EE) or plaintext
ephemeral_pub	TEXT	Recipient's ephemeral pub key (base64-SPKI)
iv	VARCHAR(64)	AES-GCM nonce for recipient
is_e2ee	BOOLEAN	True if end-to-end encrypted
sender_copy	TEXT	Sender's own ciphertext
sender_ephemeral_pub	TEXT	Sender's ephemeral pub key
sender_iv	VARCHAR(64)	AES-GCM nonce for sender
message_type	VARCHAR(20)	'direct' or 'broadcast'
file_record_id	INTEGER FK	-> file_records.id; NULL if text-only
is_deleted	BOOLEAN	Soft delete flag
deleted_at	DATETIME	UTC timestamp of deletion
timestamp	DATETIME	UTC timestamp of creation

file_records

Column	Type	Notes
--------	------	-------

id

INTEGER PK

uploader_id

INTEGER FK

-> users.id

receiver_id

INTEGER FK

-> users.id; NULL for broadcast

original_filename

VARCHAR(256)

Sanitized via secure_filename()

stored_filename

VARCHAR(256)

UUID-based name on disk

file_size

BIGINT

Original bytes

mime_type

VARCHAR(128)

Detected or Content-Type header

ephemeral_pub

TEXT

'plain-text-prototype' - not yet encrypted

TEXT

'plain-text-prototype' - not yet encrypted

visibility

VARCHAR(20)

'shared' (DM) or 'broadcast'

status

VARCHAR(20)

'processing', 'ready', or 'failed'

thumbnail_filename

VARCHAR(256)

Thumb WebP/JPG on disk

original_size

BIGINT

Pre-processing bytes

processed_size

BIGINT

Post-processing bytes

upload_tier

VARCHAR(20)

Connection tier at upload time

media_width

INTEGER

Pixels (images)

media_height

INTEGER

Pixels (images)

media_duration

REAL

Seconds (audio/video)

download_count

INTEGER

Unique-user download count

uploaded_at

DATETIME

UTC timestamp

12. Known Gaps / Not Yet Implemented

Feature	Status
File encryption at rest	FileRecord.ephemeral_pub stores 'plain-text-prototype' - files on disk are
Forward secrecy	Not implemented. Private key compromise exposes all past messages for that
Key rotation	No mechanism to rotate identity keys without clearing localStorage. key.
Broadcast E2EE	Broadcast messages are always plaintext; ratchet protocol not implemented.
Server-side integrity	Server stores ciphertext blindly - no HMAC over metadata.
Signal-style ratchet	Each message uses fresh ECIES but there is no Double Ratchet for forward
Audit log	No log of admin actions (deactivation, role changes). secrecy between messages.

13. Security Properties Summary

Property	Status	Notes
DM text confidentiality	YES	AES-256-GCM; server stores only ciphertext
DM text integrity	YES	GCM auth tag - any tamper causes decrypt failure
File confidentiality	NO	Files stored unencrypted on disk
Broadcast confidentiality	NO	Always plaintext; visible to all LAN users
Transport security	YES	TLS (self-signed) on all HTTP and WebSocket traffic
Password security	YES	PBKDF2-SHA-256 via Werkzeug
Resource authorization	YES	Per-route and per-resource checks enforced server-side
Key distribution trust	PARTIAL	Server distributes public keys - a compromised server could MITM
Perfect forward secrecy	PARTIAL	Ephemeral keys per message but no ratchet; private key
Replay protection	PARTIAL	Unique IV per message; no explicit sequence numbers

YES = implemented and verified in the current codebase. NO = not implemented. PARTIAL = partially addressed; see notes.